CONCISE INTUITIONISTIC ARITHMETIC

Protected equations

A smaller iPRA proof kernel

Construction, proof translations, and reproducible checker comparison

Prepared for Vladimir Reshetnikov

11 November 2026

Abstract

The current ConciseIPRA collection contains two useful reductions: an introduction rule whose choice is determined by the matrix, and an elimination rule whose choice is determined by the substitution source. Combining their programs alone does not establish a bi-interpretation: the available proofs of completeness have incompatible equality bootstraps. We resolve that issue by restricting public atomic formulas to equations with an iterator at both roots. Ordinary equality is represented by

$$W(s) = W(t), \qquad W(r) := I(0, r; u.0).$$

All required proof transformations can then use equality sources with a fixed iterator root. Such sources survive arbitrary numerical substitution, including substitution of zero. This permits a noncircular reconstruction of padding, numerical cases, equation induction, and both interpretation composites.

Under the repository's canonical-statement measurement convention, the three delivered complete checker profiles improve its recorded coordinate minima from 259 to 258 significant tokens, from 283 to 281 AST nodes, and from 526 to 525 normalized bytes. These minima belong to different programs. The improvement is small; no absolute minimality or literature-wide novelty is claimed. A separate strict checker validates the public statement grammar. The new equivalence proof is written mathematics, supported by executable certificates; it has not been formalized in Lean.

Baseline frozen at commit

`1517d9d79ce33f631e0f82960c849427cbba23bf`

ProveIt / Logic / PrimitiveRecursiveArithmetic / ConciseIPRA

Contents

1 The result and its precise scope **2**

2 The inherited arithmetic and native reference system **2**

3 Complete specification of the protected core **3**

3.1 Public formulas and equality 3

3.2 Auxiliary judgments and the seven rule forms 3

3.3 The executable extension $\mathcal{K}$ 4

4 The safe equality bootstrap **4**

5 Padding, numerical cases, and equation induction **5**

6 Explicit translations and the two round trips **7**

6.1 Translation from the formalized native calculus 7

6.2 Literal inclusion and the canonical round trip 7

7 Full intuitionistic logic and bi-interpretability with iPRA **8**

8 Why the protected language matters **9**

9 Executable certificate contract **9**

9.1 Trusted programs and input 9

9.2 The two combined changes 10

9.3 Soundness and the scope of completeness 10

10 Reproducible size comparison **11**

11 Validation and research status **11**

12 Using the archive **12**

A The complete byte-profile checker **12**

B A compact mathematical inventory **13**

1 The result and its precise scope

The proposed presentation is the *protected core*. It uses the same numerical terms as the current native scalar core, one public formula binder, and a deterministic introduction schema. Its essential new ingredient is a syntactic invariant on public equations, which makes two previously separate reductions compatible.

Theorem 1.1 (Main result)**.** *The protected core is bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. Both numerical interpretation domains are whole. Interpreted equality is actual equality, represented in the protected syntax by $W(x) = W(y)$. Both numerical composites are the identity up to this equality. For every public formula, both formula composites are provably equivalent to the original formula.*

The comparison is against the pinned repository state, rather than an earlier draft. Its README records the native calculus, its Lean bi-interpretation, and the existing `source_dispatch` and `guarded_core` packages [1, 4, 5]. The latter two packages supply the two reductions being combined. This report supplies the protected-equation compatibility argument, a canonical language for the combination, executable examples, and measured programs.

The source-size gain is deliberately not described as a large breakthrough. It is a strict improvement under the existing comparison contract. The strict public-grammar checker is larger than the earlier guarded strict checker; the distinction is reported in Section 10. A proof of global optimality would require substantially more than comparison with the inspected implementations.

2 The inherited arithmetic and native reference system

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in the update h is bound by the displayed iterator. The counter n, initializer a, and passive parameters are not bound by it. All substitutions below avoid capture. Put $1 = S0$. The two computation schemas are

$$I(0, a; u.h) = a, \tag{I0}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

The current Lean native system $\mathcal{N}$ has formulas $s = t, \mathsf{Q}_y B, \mathsf{D}_{x,y} B$, with ordinary readings

$$\mathsf{Q}_y B = B(0) \to \forall y B(y), \qquad \mathsf{D}_{x,y} B = \exists x (B(x, 0) \to \forall y B(x, y)).$$

Its eight constructors are hypothesis, I0, IS, equational transport, QE, DE, witnessed case introduction of D, and equation-progress introduction of Q. The repository proves transport in arbitrary formula templates and witnessed D-progress as derived rules. It also proves that these native rules present a theory bi-interpretable with iPRA [2, 3].

For orientation, its ordinary arithmetic realization uses intuitionistic equality logic, I0 and IS, numerical coverage, $(0 = 1) \to \bot$, and contextual induction for literal equations. The inherited arithmetic bridge derives predecessor, equality reflection, quantifier-free induction, pairing, and a compiler for every fixed primitive recursive definition. It proves the two composites by internal

uniqueness, not merely by standard-model agreement. The relevant written proofs and selected Lean statements are frozen in `reference/`.

We retain this arithmetic bridge. In particular, we do not assert that positive computation equations alone, without a falsity bridge, exclude a singleton ordinary structure.

3 Complete specification of the protected core

3.1 Public formulas and equality

Call a term *protected* when its outermost constructor is I. This is a syntactic condition, not a claim that its numerical value is nonzero. Every protected term keeps its I root under capture-free numerical substitution.

Definition 3.1 (Public grammar)**.** The public formulas are generated by the following clauses.

1. $p = q$ is a public formula if both p and q are protected.
2. $\mathsf{D}_{x,y}B$ is public if B is a compound public formula.
3. An atomic matrix directly below D is allowed exactly in the form

$$B(x,y) = \big(I(y,0;u.h) = I(y,0;u.k)\big). \tag{Guard}$$

   Thus $\mathsf{D}_{x,y}B$ with this matrix is public. The updates may depend on x, y, the state u, and passive parameters.

The bound variables x, y are distinct. There is no public Q constructor. Define

$$W(t) := I(0,t;u.0), \qquad s \simeq t \quad \equiv \quad W(s) = W(t), \qquad \bot_{\mathcal{P}} := (W(0) = W(1)).$$

The variables bound in W are always fresh. The equality $\simeq$ is an abbreviation, not an additional primitive relation. Since I0 proves $W(t) = t$ in the ordinary realization, its reading is actual equality throughout the entire numerical domain.

3.2 Auxiliary judgments and the seven rule forms

The proof calculus $\mathcal{H}$ also permits auxiliary equations with arbitrary term endpoints and closure resources $\mathsf{Q}_y B$. These are needed, for example, to use the computation equation $W(t) = t$ when t itself is not protected. Auxiliary equations are not additional public atomic predicates. A public theorem is a public formula having a finite derivation in the stated calculus.

Retained contexts are finite reusable lists. They may include auxiliary judgments in intermediate derivations. Q may be used with a public matrix, and D formulas in $\mathcal{H}$ obey the public grammar. The ordinary interpretation of all these auxiliary judgments is the uniform conditional reading in Section 2.

Besides I0 and IS, available in every context, the rules are precisely the following. $F(v)$ is an allowed formula or auxiliary-equation template, with the indicated instances well formed.

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ (\mathrm{Hyp}), \qquad \frac{\Gamma \vdash p = q \quad \Gamma \vdash F(p)}{\Gamma \vdash F(q)}\ (\mathrm{Tr}_I), \quad p \text{ has an } I \text{ root.} \tag{H,Tr}$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ (\mathrm{QE}),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x,y) \quad \Gamma, \mathsf{Q}_y B(b,y) \vdash A}{\Gamma \vdash A}\ (\mathrm{DE}).$$

In DE, b is fresh for the retained context, conclusion, and fixed parameters. The sole D-introduction schema is

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is an equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{DI}$$

Here the conclusion must be public, z is fresh for Γ, t and the fixed parameters, and the displayed local resource is discharged. At an equation matrix the guard requirement (Guard) applies. The certificate does not choose between cases and progress; the matrix root determines the premise.

Counting the two computation schemas separately gives seven forms: Hyp, I0, IS, Tr, QE, DE, and DI. There are no primitive reflexivity, successor-injectivity, separation, coverage, conventional connective, or induction axioms. Structural rules are admissible operations on finite derivations, not additional certificate instructions.

3.3 The executable extension $\mathcal{K}$

The compact programs permit more administrative syntax. In particular, their transport may use any source other than the literal zero, and their raw D formulas need not pass the public guard test. Introduction is still determined by the matrix root. This extension is denoted $\mathcal{K}$; every $\mathcal{H}$ inference has an encoding in it.

Every such raw inference is sound under the *same ordinary conditional reading*: equation-progress introduction is sound by equation induction under its conditional base; compound-case introduction is sound by numerical coverage. No progressive alternative meaning of D is necessary for our soundness argument. The restrictions on public formulas are needed for reverse translation and formula composites. We do not claim completeness or a bi-interpretation for every unrestricted raw statement.

4 The safe equality bootstrap

The key invariant is stronger than "the source is not zero": every transport source used in the completeness proof has an I root. This makes the proof transformations stable under substitution.

Lemma 4.1 (Structural operations)**.** *Weakening, exchange, contraction, fresh renaming, numerical substitution, and ordinary-assumption cut are admissible in $\mathcal{H}$.*

Proof. Induct on derivations, renaming eigenparameters before substitution or cut. Public guards retain their shape because their bound counter is not substituted by free-parameter substitution. An I-headed transport source retains its head. All other rule side conditions are the usual explicit freshness conditions. Cut replaces a hypothesis use by its supplied derivation. This proof would fail for a rule merely requiring a variable source to be syntactically nonzero; that weaker invariant is not being used here. □

Lemma 4.2 (Safe equality operations)**.** *The following operations have derivations whose transport sources all have I roots.*

1. *Reflexivity $t = t$ for any term t.*
2. *From $p = q$, symmetry $q = p$, when p is protected.*
3. *From $r = p$ and $p = q$, transitivity $r = q$, when p is protected.*
4. *From $p = q$, the equation $f(p) = f(q)$, when p is protected and f is a fixed numerical template.*

Consequently ordinary equality algebra is available between protected endpoints.

Proof. Put $a = W(t)$. I0 gives $a = t$. Use this equation as both premises of transport with template $v = t$; the result is $t = t$. For symmetry transport $p = p$ along $p = q$ in the template $v = p$. For transitivity use the template $r = v$. For congruence transport reflexivity of $f(p)$ in the template $f(p) = f(v)$. The last operation may produce a nonprotected endpoint; such an endpoint is never silently used as a transport source in later arguments. □

Lemma 4.3 (Joining at zero)**.** *For protected p, q, from $p = 0$ and $q = 0$ one can derive $p = q$ using only I-headed transport sources.*

Proof. Let $w = W(0)$. From $p = 0$, transport reflexivity $W(p) = W(p)$ on the left to obtain $w = W(p)$. Its safe symmetry gives $W(p) = w$. Now use the computation $W(p) = p$ to obtain $p = w$. Similarly obtain $q = w$. All the endpoints p, q, w are protected, so safe symmetry and transitivity yield $p = q$. At no point is $0 = p$ used as a transport major. □

Lemma 4.4 (Lifting an arbitrary equality)**.** *For arbitrary terms s, t and a numerical template $a(v)$,*

$$W(s) = W(t) \;\vdash_{\mathcal{H}}\; W(a(s)) = W(a(t)).$$

Consequently the native equational transport rule translates under $s = t \mapsto W(s) = W(t)$.

Proof. Congruence on the given protected equation gives $W(a(W(s))) = W(a(W(t)))$. The two computation equations $W(s) = s$ and $W(t) = t$, whose sources are protected, give

$$W(a(W(s))) = W(a(s)), \qquad W(a(W(t))) = W(a(t)).$$

Every endpoint in these three displayed equations has a fixed W root. They can therefore be reversed and composed safely, proving the claim. Apply this construction to both numerical sides of an old equation template, and compose with the translated minor premise. The argument still works when s is literally zero. □

This is the compatibility step absent from a bare composition of the two old programs. It replaces a demand for unrestricted zero-source symmetry by a translation of all public equalities into protected ones.

5 Padding, numerical cases, and equation induction

Set $C(v, a, b) = I(v, a; u.b)$, with the state variable unused. For a protected equation $E = (p = q)$, define

$$G_v E = (C(v, 0, p) = C(v, 0, q)), \qquad L(E) = \mathsf{D}_{a,v} G_v E,$$

where a, v are fresh for E. Let $P(E) = L(E)$ for an equation, and $P(B) = B$ for a compound public formula. Thus $P(B)$ is always compound and public.

Lemma 5.1 (Guard endpoints and padding)**.** *For every protected equation E,*

$$\vdash G_0E, \qquad G_{Sz}E \dashv\vdash E, \qquad L(E) \dashv\vdash E.$$

These interderivabilities are uniform under additional retained context.

Proof. The two sides of G_0E compute to zero. They are protected, so Lemma 4.3 joins them. At a successor, the two sides compute to p and q. Both are protected, so safe equality algebra gives the endpoint equivalence in both directions. Under E, atomic DI proves $L(E)$: its successor goal follows from E, independently of its local induction assumption. Conversely, DE on $L(E)$ supplies $\mathsf{Q}_v G_v E$. Feed its proven zero instance and use QE at 1, then remove the successor guard. □

For protected equations $U = (p = q)$, $V = (r = s)$, put

$$M_v(U, V) = (C(v, p, r) = C(v, q, s)), \qquad U \Rightarrow_g V := \mathsf{D}_{a,v} L(M_v(U, V)).$$

The outer matrix is compound. Endpoints and padding show that this macro has implication introduction and elimination between protected equations, using compound-case DI and DE followed by QE at 1.

Lemma 5.2 (Numerical cases)**.** *For every public formula $A(y)$,*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}$$

is admissible when z is fresh for the retained context and fixed parameters.

Proof. If A is compound, introduce $\mathsf{D}_{a,y}A(y)$ with unused a by cases, ignoring its local base assumption. Eliminate it, supply the given base, and use QE at t. For an equation apply this compound case construction to $L(A(y))$, padding both premises and unpadding the conclusion. □

Theorem 5.3 (Contextual induction for protected equations)**.** *For every equation $E(y)$ whose two sides have fixed I roots,*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{PEI}$$

is admissible, with the usual eigenvariable condition.

Proof. Put $J(y) = G_y E(y)$. We need its progress step without retaining a case-variable-dependent assumption during numerical cases. Use the compound formula $\Phi(y) = J(y) \Rightarrow_g E(Sy)$.

For $y = 0$, ignore the implication's local assumption, take the given $E(0)$, and specialize the given step at zero to obtain $E(1)$. For $y = Sw$, the assumption $J(Sw)$ gives $E(Sw)$; the step specialized at Sw gives $E(SSw)$. Both branch derivations have retained context Γ, independent of the case variable. Lemma 5.2 yields $\Phi(z)$. Under $J(z)$, implication elimination therefore yields $E(Sz)$, and the successor guard gives $J(Sz)$.

Atomic DI now proves $\mathsf{D}_{a,y}J(y)$ with unused a. DE and the theorem $J(0)$ supply $J(Sw)$, hence $E(Sw)$, for fresh w. Numerical cases combine these successor instances with the original $E(0)$, yielding $E(t)$.

The substitutions into the supplied step are legitimate by Lemma 4.1: its derivation is in $\mathcal{H}$, whose transport sources retain an I root. This is the point at which a naive appeal to arbitrary nonzero-source derivations would be circular. □

6 Explicit translations and the two round trips

6.1 Translation from the formalized native calculus

Define T on finite native formulas, with all new binders fresh:

$$\begin{aligned} T(s = t) &= W(s) = W(t), \\ T(\mathsf{Q}_y B) &= \mathsf{D}_{a,y} P(TB) \quad (a \text{ unused}), \\ T(\mathsf{D}_{x,y} B) &= \mathsf{D}_{x,y} P(TB). \end{aligned}$$

Numerical terms are unchanged. This translation is effective, commutes with capture-free numerical substitution up to renaming, and always produces public formulas. Translate contexts pointwise.

Write $\mathrm{Cl}_y\, B = \mathsf{D}_{a,y} P(B)$ for the translated closure. From $\mathrm{Cl}_y\, B$ and $B(0)$, DE, padding, QE, and unpadding derive $B(t)$. From $\Gamma, B(0) \vdash B(Sz)$, compound-case DI proves $\mathrm{Cl}_y\, B$. These statements use the endpoint interderivabilities of Lemma 5.1.

Theorem 6.1 (Native proof translation)**.**

$$\Gamma \vdash_{\mathcal{N}} A \quad \Longrightarrow \quad T\Gamma \vdash_{\mathcal{H}} TA.$$

The transformation is effective on finite derivations and uses only I-headed transport sources.

Proof. Induct on the native derivation. Hypothesis use is immediate. Each computation equation has an I-headed source, so safe congruence through W gives its translated computation. Native equational transport is Lemma 4.4, applied to both sides. QE uses the translated closure elimination just described.

For native case DI, unpad its translated local base, apply the translated premise, and pad the successor result. The target matrix $P(TB)$ is compound, so the root-determined rule uses cases.

For native Q-progress on E, its translated step is a step for the protected equation TE. Under the translated base, use Theorem 5.3 at Sz. Pad the result and use compound case introduction to prove $\mathrm{Cl}_y\, TE$.

For native DE, its translated major opens the raw resource $\mathsf{Q}_y P(TB(b, y))$. This resource supplies its corresponding public closure $\mathrm{Cl}_y\, TB(b, y)$: introduce that closure by cases, and use the retained raw Q resource at the successor. Cut this public closure into the recursively translated minor derivation, then close DE. Freshening all auxiliary binders preserves the original eigenvariable conditions. □

6.2 Literal inclusion and the canonical round trip

Let J send a public formula to the identical native Eq/D formula. Every $\mathcal{H}$ inference is valid in $\mathcal{N}$: unrestricted formula transport and witnessed equation-progress are derived native rules. This provides the reverse proof interpretation.

Lemma 6.2 (Public round trip)**.** *For every public formula A,*

$$A \dashv\vdash_{\mathcal{H}} T(JA).$$

Proof. Induct on the number of D binders. For an equation $p = q$, both endpoints are protected. The computations $W(p) = p$ and $W(q) = q$ therefore have protected endpoints, and safe equality algebra proves $p = q \dashv\vdash W(p) = W(q)$.

For a D formula with compound matrix, use compound binder congruence: eliminate the source at a fresh witness; introduce the target by cases; convert its base backwards, use the opened closure at the successor, and convert the result forwards. Matrix interderivability is given by the external induction hypothesis. All conversions use safe derivations.

It remains to treat $\mathsf{D}_{x,y}E$ with an atomic guard matrix. Its zero instance $E(x,0)$ is a theorem, uniformly in x, by Lemma 4.3. Its translation is $\mathsf{D}_{x,y}L(TE)$. Forwards, DE opens $\mathsf{Q}_y E(b,y)$. Supply its proven zero instance, obtain each successor instance, translate the protected equation by the atomic equivalence, and pad. Compound-case DI at witness b then proves $\mathsf{D}_{x,y}L(TE)$. Backwards, DE opens $\mathsf{Q}_y L(TE(b,y))$. Pad the translated zero theorem and use QE at Sz. Unpadding and the atomic equivalence give $E(b,Sz)$. Atomic-progress DI, ignoring its local induction assumption, proves $\mathsf{D}_{x,y}E$. Close DE in both directions. This case does not assume a nonexistent atomic case-introduction rule. □

The other native formula composite follows by external induction: native equality proves $W(t) = t$, native padding is equivalent to its matrix, and the unused-witness D represents Q. Thus $JT(A) \dashv\vdash_{\mathcal{N}} A$ for every native formula, including those with Q constructors.

Corollary 6.3 (Equality of canonical consequence relations)**.** *For finite public contexts* Γ *and public* A*,*

$$\Gamma \vdash_{\mathcal{H}} A \quad \Longleftrightarrow \quad J\Gamma \vdash_{\mathcal{N}} JA.$$

For empty external context and closed canonical goals, the same equivalence holds for finite accepting compact certificates under the decoding contract in Section 9.

Proof. Left to right is literal native soundness. For the converse, translate the native proof by Theorem 6.1, then use the public round trips on the retained assumptions and conclusion, with cut. The compact extension is native-sound, and every $\mathcal{H}$ proof has an encoding, so its canonical consequences lie between these equal consequence relations. □

7 Full intuitionistic logic and bi-interpretability with iPRA

There is no restriction to the equational or quantifier-free fragment of iPRA. To specify the logical interface explicitly, let enc be the existing native encoding of ordinary formulas, and define

$$\mathrm{enc}_{\mathcal{P}}(F) := T(\mathrm{enc}(F)).$$

This is a finite structural macro expansion, not an appeal to a provability oracle. The native encoding constructs implication, conjunction, disjunction, existence, and universality from its selector and gate operations [2]. For example, its macros are

$$\begin{aligned} A \to B &:= \mathsf{Q}_y\, \mathrm{sel}(y, A, B), \\ A \wedge B &:= \mathsf{Q}_y\, \mathrm{gate}(y, \mathrm{sel}(\mathrm{par}(y), A, B)), \\ \exists x A &:= \mathsf{D}_{x,y}\, \mathrm{gate}(y, A(x)), \\ A \vee B &:= \exists v\, \mathrm{sel}(v, A, B), \\ \forall x A &:= A(0) \wedge \mathsf{Q}_y A(y). \end{aligned}$$

Here the formula selector and gate are defined by recursion on finite formula syntax, using the numerical selector C, binder congruence, and padding. Applying T removes the public Q constructors

and protects all atoms. All introduction and elimination rules transfer by Theorem 6.1. Thus the displayed complete native macro implementation in `reference/Lean/NativeLogic.lean`, composed with our explicit T, specifies all ordinary logical operations without adding axioms or trusted checker code.

For numerical terms, the forward map to iPRA uses the inherited primitive-recursive implementation of each fixed iterator. The reverse map uses its scalar compiler for each fixed primitive-recursive definition. No universal evaluator over arbitrary function codes is postulated. The inherited round-trip equalities translate by T to protected equalities, so they hold internally in the new theory.

The domain predicate in both directions is the whole domain. The numerical identity graph is $x = y$ in the ordinary realization and $W(x) = W(y)$ in the protected presentation. Since I0 identifies $W(t)$ with t ordinarily, there is no quotient, retract, standard-numeral restriction, or hidden change in equality. The public round trip and the inherited native/iPRA round trips prove both full formula composites. Finally,

$$\mathcal{P} \vdash W(0) = W(1) \quad \Longleftrightarrow \quad \mathcal{N} \vdash 0 = 1 \quad \Longleftrightarrow \quad \text{iPRA} \vdash \bot.$$

This proves Theorem 1.1. It is a metatheoretic equiconsistency statement, not a claim that either theory proves its own consistency.

8 Why the protected language matters

The words “not literally zero” alone do not supply the structural invariant needed in the proof. A variable source is permitted by that test, but substituting zero can turn its use into a forbidden rule instance. Moreover, guarding an arbitrary equation can require inserting wrappers around a zero endpoint; unrestricted equality can do this, whereas restricted forward transport cannot simply assume it. Using one old completeness argument inside the other without checking these dependencies would be circular.

Protected atoms resolve both issues. All equality information that must later act as a major is represented at stable iterator roots. Zero computations are joined through $W(0)$, and arbitrary native equalities are moved through the sandwich in Lemma 4.4. Numerical substitution is first proved in $\mathcal{H}$, before it is used in the induction bootstrap.

There is a related distinction in the upstream README. The statement that substitution is not *admissible* in its already-equivalent ZPrf system is too strong. Its equivalence theorem `zprf_iff_nprf` transports native substitution to ZPrf. What fails is direct substitution into the same derivation tree without repairing rule instances. This correction does not solve the guarded composition by itself, since equivalence of that new composition has not yet been established at that stage. Our stable-head proof avoids using such a corollary prematurely.

9 Executable certificate contract

9.1 Trusted programs and input

Each compact profile is a complete standalone Python module importing only the standard JSON parser. It reads a JSON array from standard input. The last element is the closed goal; preceding instruction packets are consumed backwards. Success is normal process exit with exactly `ACCEPT`

followed by a newline. An exception, nonzero exit, timeout, or resource exhaustion is not acceptance. Unused prefix data is inert, as in the inherited protocol. No producer, normalizer, theorem database, or Python text supplied by the certificate is executed.

The arity encoding is inherited. At numerical positions, unary arrays mean successor and ternary arrays mean iterator. At formula positions, unary arrays mean D, binary arrays mean equality, and ternary arrays mean Q. A canonical Q has three copies of its body. The opener adjusts depth by $5 -$ array arity. Protected slots have width two; numerical root depth is three and formula root depth zero. The full sorted interpretation, including zero and truth aliases, is supplied in `reference/inherited_arity_checker.tex`.

The public grammar adds two explicit requirements to this convention: both numerical children of every public equation have ternary iterator roots; and an atomic D matrix has both sides beginning with `[5,0]`, the inherited encoding of the displayed guard. The strict checker enforces the closed canonical goal grammar before running the same kernel. It does not reject sound administrative intermediate resources merely because they are not public formulas.

9.2 The two combined changes

The source-dispatch matcher is

```
m == ([s, u] if s else [t] * 3)
```

so the source determines whether the major is equality or Q. The root-determined introduction uses

```
(m or not len(a) % 2) and k
```

so an atomic matrix takes its progress instance and a compound matrix takes its zero instance. The latter removes the old comparison with a certificate-supplied choice. The complete byte profile appears in Appendix A.

Canonical $\mathcal{H}$ transport sources have a ternary iterator root and are therefore truthy in this encoding. The source matcher accepts them. Atomic progress and compound case introduction match the new root test. The inherited computation normalization supplies the restricted computation packets while preserving their conclusions; public guards need not be changed. A producer and concrete safe equality normalization are delivered, but are outside the trusted program.

9.3 Soundness and the scope of completeness

Soundness of a finite accepting trace uses the inherited opening, scope, and sorted-alias lemmas. The source-selected equality and Q major are still verified. The new introduction chooses one of the two already-sound Choice-DI rules. Thus every accepted raw proof has a native-provable ordinary reading.

For a canonical public goal, Corollary 6.3 turns that sound reading back into a protected proof. Conversely every finite $\mathcal{H}$ proof can be encoded with fresh integer parameters and the inherited computation expansions. This proves equality of canonical theorems in the ideal unbounded implementation. Physical Python stack and memory limits can reject large valid proofs; no completeness claim over a fixed finite resource bound is made.

The compact profiles intentionally do not validate the exact public goal grammar. Their record is measured under the same canonical-statement convention used by the source packages. The strict

profile supplies the additional public-boundary check and is counted in its entirety. We do not omit that validator while claiming an exact-public-grammar program of 525 bytes.

10 Reproducible size comparison

The measurements use CPython 3.12.14, the version used by the current repository. Normalized source is `ast.unparse` output with a final newline. Comments and non-executable docstrings do not contribute to significant-token or executable-AST counts. Every selected compact program includes JSON input, scope traversal, proof checking, and output. Gzip uses level 9 and `mtime=0`; it is an informational coordinate, not an optimization objective.

Profile	Program	Tokens	AST nodes	Bytes	Gzip-9
Token	prior source-dispatch	259	289	572	339
	protected core	**258**	287	571	339
AST	prior source-dispatch	263	283	607	348
	protected core	262	**281**	606	348
Byte	prior source-dispatch	266	305	526	330
	protected core	265	303	**525**	331

Thus the coordinate envelope decreases from $(259, 283, 526)$ to $(258, 281, 525)$. It is not the size triple of one program. The token and AST profiles improve their corresponding old programs in all three principal coordinates; the byte profile does too. Compression of the byte profile increases by one byte, which is reported rather than hidden.

The prior guarded profiles have minima 260 tokens, 286 AST nodes, and 527 normalized bytes. Their combination with source dispatch therefore improves that frontier as well. This is a comparison with the recorded current implementations, not a lower-bound theorem for every possible proof checker.

The strict protected checker has 505 significant tokens, 582 AST nodes, 1,251 normalized bytes, and 582 gzip-9 bytes. The previous guarded strict checker has 1,208 normalized bytes. There is therefore *no strict-parser size improvement* in this comparison. The new global protected-atom check has a real cost. All raw and normalized measurements, frozen source hashes, and generated gzip files are included in the evidence directory.

11 Validation and research status

The reproduction suite passed all 608 executions. It checks 5 candidate programs and 6 frozen comparison programs, using 11 named canonical examples and 22 negative fixture families. Candidate executions cover both ordinary Python and Python with optimization enabled. The examples include both wrapped computation schemas, protected reflexivity, compound introduction and elimination, both padding directions, a 564-instruction guarded induction certificate, and a guarded implication expressing explosion from the protected contradiction. All generated named derivations pass the guard and source audits.

The induction example is not a vacuous use of its induction hypothesis: the supplied audit records an actual use and verifies that the safe normalization introduces no literal-zero transport source. The certificates preserve the guarded matrices. The contradiction example uses the public contradiction

$W(0) = W(1)$, rather than relying on an unprotected equation $0 = 1$ as an interchangeable raw assumption.

These executable results verify the delivered programs and examples. They do not prove the universal interpretation theorem by finite testing. The theorem is supported by the explicit structural proof above and an independent written review supplied in the archive. The inherited native/iPRA bridge is formalized in the cited repository; its Lean build was not rerun here. The protected-core translation and the small raw Python kernel have not been verified in Lean or Rocq.

12 Using the archive

From the archive root, run

```
python3 reproduce.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The Python reproduction uses the standard library. It regenerates certificates, audits them, executes positive and negative controls, and reports size measurements. A TeX installation with the packages listed in the preamble is needed only to rebuild the PDF.

File	Purpose
`code/checker_bytes.py`	Smallest byte profile
`code/checker_tokens.py`	Smallest token profile
`code/checker_ast.py`	Smallest AST profile
`code/readable_checker.py`	Expanded control flow for review
`code/strict_checker.py`	Public-goal validation plus kernel

Local-scope and global-scope certificate files are labeled separately; they are not claimed interchangeable.

The reproducibility manifest identifies the precise original files and Git blobs used in the comparison. The inherited references retain their provenance. The novel contribution is the protected language, stable-source completeness proof, compatible combined kernels, and their validation. No repository files were changed remotely.

A The complete byte-profile checker

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d else d %
      2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t,
     2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or not len(a) % 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

B A compact mathematical inventory

Component	Specification
Numerical terms	$x, 0, S(t), I(n, a; u.h)$; one numerical sort.
Public atoms	$p = q$, both endpoints with a fixed iterator root.
Public binder	$\mathsf{D}_{x,y}B$; an atomic matrix must have the zero guard.
Equality macro	$s \simeq t$ abbreviates $W(s) = W(t)$.
Contradiction	$W(0) = W(S0)$.
Auxiliary resources	Arbitrary equations and conditional closures Q.
Axioms	The two computation schemas I0 and IS.
Rules	Hyp, stable-source Tr, QE, DE, and deterministic DI.
Derived structure	Numerical substitution and cut; no structural kernel rules.
Derived arithmetic	Protected equation induction, then the translated native arithmetic bridge.
Domain composite	Whole numerical domain; identity graph represented by $\simeq$.
Formal status	New written proof and executable regression evidence; inherited Lean bridge.

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. README at commit `1517d9d79ce33f631e0f82960c849427cbba23bf`. Pinned repository version.

[2] ProveIt contributors. *Native.lean, NativeStruct.lean, NativeEq.lean, NativeLogic.lean*. Same pinned commit, `Lean/ConciseIPRA/`. Selected exact sources are frozen in the accompanying `reference/Lean/` directory.

[3] ProveIt contributors. *NativeBiInterpretation.lean* and the inherited ordinary arithmetic bridge. Same pinned commit. The former states the faithful proof translations, both full formula round trips, and equiconsistency; the latter is reproduced with the predecessor, equality-reflection, and primitive-recursion arguments in `reference/`.

[4] ProveIt contributors. *Source-dispatch research package* and *NativeZeroSource.lean*. Same pinned commit, `Research/source_dispatch/` and `Lean/ConciseIPRA/NativeZeroSource.lean`. Pinned source-dispatch package.

[5] ProveIt contributors. *A guarded core with a deterministic introduction rule*. Same pinned commit, `Research/guarded_core/`. The guarded bootstrap is inherited; the restriction to protected atoms and the stable-source compatibility argument are the new work here. Pinned guarded-core package.